

Лекция 5. RAG: Retrieval-Augmented Generation

Расширенный конспект, анализ транскрибации, корректировки и практический план реализации

Главная идея: RAG не обучает модель заново и не меняет ее веса. Он достает релевантные фрагменты из вашей базы знаний во время inference и добавляет их в prompt, чтобы ответ был основан на внутренних документах, ссылках и фактах, а не только на общей памяти LLM.

Источники: транскрибация лекции 5 и приложенные скрины презентации. Документ намеренно расширен по сравнению с первой короткой версией: добавлены детали pipeline, таблицы различий, корректировки терминов, анти-паттерны, метрики качества и практический MVP для задания недели.

1. Контекст лекции и зачем нужен RAG

Пятая лекция продолжает цепочку предыдущих недель: сначала были базовые LLM-параметры и промптинг, затем context management, memory/state, MCP и tools. Теперь фокус смещается на RAG - Retrieval-Augmented Generation.

RAG нужен там, где общей модели недостаточно. LLM может хорошо рассуждать и писать текст, но она не знает ваши внутренние регламенты, Confluence, проектные решения, корпоративные договоренности, историю чатов, актуальные версии документов и локальные правила команды.

Смысл RAG: перед генерацией ответа найти релевантные документы или фрагменты документов, добавить их в prompt и заставить модель отвечать на основе найденного контекста.

- Без RAG модель отвечает “по среднему интернету”.
- С RAG модель отвечает с опорой на вашу базу знаний.
- Хороший RAG должен возвращать не просто красивый ответ, а ответ с источниками и проверяемыми ссылками.

2. Терминология: RAG, а не RUG/RAC/пак

В транскрибации термин распознавался по-разному: RUG, RAC, “пак”. Корректный термин - RAG, Retrieval-Augmented Generation.

Дословно это можно понимать как “генерация, дополненная поиском/извлечением”. Сначала система достает релевантные знания, потом LLM генерирует ответ с учетом этих знаний.

2.1. Простая формула RAG

Шаг	Что происходит	Зачем это нужно
Retrieval	Поиск релевантных chunks в документах, базе знаний или векторном хранилище.	Найти факты, которые модель должна использовать.
Augmentation	Добавление найденных chunks в prompt вместе с вопросом пользователя.	Дать модели внешний контекст прямо во время inference.
Generation	LLM формирует ответ, желательно с источниками.	Получить человеческий, связный и проверяемый результат.

2.2. Важное ограничение

RAG не “загружает знания в мозг модели” навсегда. Он не меняет веса модели. Если в следующий запрос не подмешать нужные документы заново, модель снова будет отвечать только из своего общего контекста и текущего prompt.

3. Какие проблемы решает RAG

В лекции выделяется несколько практических проблем, которые возникают при работе с корпоративными или личными знаниями.

3.1. Общая модель не знает ваши внутренние данные

Модель обучалась на больших публичных корпусах: книгах, вебе, GitHub, Reddit, форумах, документации и других источниках. Но она не обучалась на вашем внутреннем Confluence, Jira, Notion, локальных регламентах, чатах команды или семейной базе документов.

3.2. Контекст большой, смешанный и неполный

Даже если все документы вручную сложить в Markdown, это не решит проблему полностью. Документов может быть слишком много, они могут быть разных лет, с разной структурой, дубликатами, противоречиями и устаревшими версиями.

- Контекст может не помещаться в окно модели.
- Поиск по сырому Markdown может быть плохим.
- В базе знаний могут лежать старые и новые версии одного процесса.
- Разные команды могут описывать один процесс по-разному.

3.3. Ложноположительные ответы опаснее явных ошибок

Лектор правильно подчеркивает: иногда неправильный ответ не так страшен, как ответ, похожий на правильный. Если пользователь некомпетентен или торопится, он может принять красивую галлюцинацию за факт.

Поэтому RAG должен не просто “что-то найти”, а помогать проверять ответ: показывать источники, даты, названия документов, фрагменты и confidence/score, если это возможно.

4. RAG vs Prompt Engineering vs MCP vs Fine-tuning

В лекции важно различать несколько способов улучшения поведения LLM. Они не заменяют друг друга, а часто используются вместе.

Подход	Что оптимизирует	Когда использовать	Что не решает
Prompt engineering	Инструкции, формат ответа, роль, ограничения, стиль.	Когда нужно быстро управлять поведением модели без внешней базы.	Не дает надежного доступа к большому корпусу знаний.
RAG	Контекст: какие факты модель получает во время inference.	Когда нужно отвечать по базе знаний, документам, Confluence, policy, договорам.	Не меняет стиль/навык модели и не заменяет бизнес-логику.
MCP/tools	Действия и доступ к внешним системам/API.	Когда агент должен не только знать, но и делать: календарь, CRM, Jira, файловая система.	Не является сам по себе системой поиска знаний.
Fine-tuning	Веса модели: как она склонна отвечать или выполнять типовые задачи.	Когда нужен устойчивый стиль, формат, доменная манера, классификация, специфическое поведение.	Не лучший способ хранить часто меняющиеся факты.

4.1. Главная разница RAG и MCP

MCP обычно про инструменты и внешние действия: сходить в календарь, вызвать API, получить список задач, создать событие, запросить внешний сервис. RAG в первую очередь про знания: найти релевантные документы и включить их в prompt.

На практике они могут комбинироваться. Например, MCP-tool достает документ из Confluence, а RAG-компонент индексирует, ищет и ранжирует chunks по этому документу.

4.2. Главная разница RAG и fine-tuning

Fine-tuning меняет веса или адаптацию модели. RAG не меняет веса. RAG дает модели “шпаргалку” во время ответа. Поэтому RAG лучше подходит для фактов, которые часто меняются: policies, процессы, цены, регламенты, актуальная документация.

5. Правильный pipeline RAG-системы

Хороший RAG - это не “засунуть документы в векторную базу”. Это pipeline из нескольких стадий. Если сломать одну стадию, ответ может стать хуже, даже если модель сильная.

1. Сбор данных: PDF, Markdown, HTML, Confluence, Notion, GitHub, Jira, Google Drive, чаты.
2. Очистка и нормализация: убрать мусор, навигацию, повторяющиеся футеры, битую разметку, OCR-ошибки.
3. Chunking: разбить документы на осмысленные фрагменты.
4. Embeddings: превратить chunks и запрос пользователя в векторы.
5. Vector store: сохранить векторы вместе с metadata.
6. Retrieval: найти top-k похожих chunks.
7. Reranking: переупорядочить найденные chunks более точной моделью.
8. Prompt building: собрать вопрос + найденные chunks + правила ответа.
9. Generation: получить ответ от LLM.
10. Citations/evidence: показать источники и фрагменты, на которых основан ответ.
11. Evaluation/feedback: измерить качество и улучшать pipeline.

5.1. Что важно хранить вместе с chunk

Metadata	Зачем нужна
document_id	Понимать, из какого документа пришел chunk.
title	Показывать пользователю нормальный источник, а не случайный UUID.
url/path	Дать ссылку на исходник.
created_at / updated_at	Фильтровать устаревшие документы.
section heading	Понимать место chunk внутри документа.
access level	Не показывать пользователю то, к чему у него нет прав.
chunk_index	Восстанавливать соседние chunks и порядок.

6. Embeddings: что это и что тут важно

Embedding model превращает текст в числовой вектор. Похожие по смыслу тексты должны иметь близкие векторы. По этим векторам система ищет chunks, которые похожи на вопрос пользователя.

6.1. Generative model и embedding model - разные роли

Модель	Роль	Пример использования
Embedding model	Кодирует текст в vector.	Векторизовать документы и запрос пользователя.
Generative model	Генерирует ответ на естественном языке.	Сформулировать ответ на основе найденных chunks.
Reranker / cross-encoder	Сравнивает query и chunk точнее, но медленнее.	Отобрать лучшие chunks из top-20/top-50.

6.2. Корректировка про Ollama

В транскрибации звучит формулировка, будто Ollama - "лучшая модель". Технически корректнее так: Ollama - это локальный runtime/model server, через который можно запускать разные модели, включая embedding-модели. Сама Ollama не является одной конкретной embedding-моделью.

Сильная сторона локального варианта: данные могут не покидать компьютер или закрытый контур компании. Это важно для privacy и корпоративных документов.

6.3. Нормализация векторов

В лекции нормализация описана упрощенно как деление на максимальное значение. Для embedding search чаще встречается L2-нормализация, когда вектор приводится к единичной длине, чтобы cosine similarity корректно сравнивала направления векторов. На практике многие embedding-библиотеки уже возвращают нормализованные векторы или умеют нормализовать их автоматически.

7. Chunking: как резать документы

Chunking - одна из самых важных частей RAG. Плохой chunking может испортить ответ даже при хорошей модели и хорошем vector store.

Тип chunking	Как работает	Плюсы	Минусы
Fixed-size	Режем по N токенов/символов.	Просто реализовать.	Может разорвать предложение или смысл.
Fixed-size + overlap	Каждый следующий chunk частично перекрывает предыдущий.	Снижает потерю контекста на границах.	Больше chunks, больше затрат.
Semantic	Режем по разделам, заголовкам, абзацам, смысловым блокам.	Лучше сохраняет смысл.	Сложнее реализовать и тестировать.
Metadata-aware	Учитываем title, section, document type, дату, автора.	Лучше фильтрация и объяснимость.	Нужно чистить и хранить metadata.

7.1. Overlap

Overlap нужен, чтобы важный смысл не потерялся на границе chunks. Например, если первая часть предложения попала в один chunk, а продолжение - в другой, retrieval может найти только половину смысла. Перекрытие снижает этот риск.

Типовые значения: chunk size 500-1000 токенов, overlap 50-200 токенов. Но универсального числа нет: нужно экспериментировать на своих документах, модели и задаче.

7.2. Что ломает chunking

- PDF с колонками, футерами и номерами страниц, которые попадают в каждый chunk.
- Сканы без нормального OCR.
- Таблицы, где строки теряют заголовки колонок.
- Markdown без заголовков или с кривой структурой.
- Дубликаты документов за разные годы.
- Очень короткие chunks без смысла или слишком длинные chunks с несколькими темами сразу.

8. Retrieval: как система ищет документы

На этапе retrieval вопрос пользователя тоже превращается в embedding. Затем vector store ищет chunks с максимальной близостью к этому query-vector.

8.1. Тор-k и фильтры

Обычно система достает top-k chunks: например, top-5, top-10, top-20 или top-50. Чем больше k, тем выше шанс найти нужный факт, но тем больше шум и расход контекста.

- Если k слишком маленький - можно не достать нужный документ.
- Если k слишком большой - prompt забивается лишним шумом.
- Metadata filters помогают сузить поиск: дата, проект, команда, тип документа, права доступа.

8.2. Пример из лекции: policy code review

Без RAG модель может ответить: “обычно достаточно одного approval и ревью в течение недели”. Это может быть правдой где-то в интернете, но не в вашей компании.

С RAG система должна найти внутренний документ code review policy и ответить примерно так: “В нашем проекте требуется минимум 2 approval до merge, а review должен быть выполнен в течение 24 часов”, плюс дать ссылку на источник.

9. Reranking: зачем нужен второй отбор

Retrieval по embedding быстро находит похожие chunks, но похожесть не всегда означает релевантность. Например, запрос про “CI/CD для Kotlin Multiplatform” может найти старый обзор CI/CD-инструментов за 2023 год, потому что там тоже есть похожие слова.

9.1. Bi-encoder vs cross-encoder

Подход	Как работает	Плюсы	Минусы
Bi-encoder retrieval	Query и chunks кодируются отдельно в embeddings, потом сравниваются.	Быстро, масштабируемо, подходит для первичного поиска.	Может вернуть похожее, но не отвечающее на вопрос.
Cross-encoder reranking	Модель смотрит query и chunk вместе и оценивает релевантность пары.	Точнее ранжирует top-k.	Медленнее и дороже, обычно применяется только к top-20/top-50.

Практически: сначала быстро достаем top-20 chunks, потом reranker выбирает top-3/top-5, которые реально стоит положить в prompt.

10. Как измерять качество RAG

RAG нужно оценивать не “на глаз”, а через метрики. Иначе можно получить красивый демо-ответ, который разваливается на реальных документах.

Метрика	Вопрос	Как проверить
Context relevance	Нашли ли мы правильные chunks?	Оценивать top-k вручную, thumbs up/down, golden dataset.
Faithfulness	Ответ основан на найденном контексте или модель придумала?	Проверять, есть ли утверждения в sources.
Answer correctness	Финальный ответ правильный?	Сравнить с эталонным ответом или экспертной оценкой.
Source quality	Источники актуальные и авторитетные?	Проверять дату, владельца документа, статус deprecated.

10.1. Минимальный evaluation set

Для учебного проекта достаточно 10-20 контрольных вопросов по своей базе знаний. Для каждого вопроса стоит заранее указать: какие документы должны найтись, какой ответ правильный, какие источники считаются допустимыми.

11. Где RAG применяется

В лекции приводятся несколько направлений, где RAG дает реальную пользу.

- Корпоративные базы знаний: Confluence, Notion, внутренние wiki, регламенты, policies.
- Чаты и переписки: Slack, Telegram, Teams, Discord, если есть право и смысл индексировать.
- Специализированные агенты: медицина, лабораторная диагностика, физика, механика, юриспруденция, образование.
- Глобальный анализ трендов: длинные исследования по большому массиву документов.
- Цифровой двойник: персональная история, заметки, переписки, звонки, видео и другие личные данные.

11.1. Privacy и закрытый контур

Для корпоративных и личных данных важно, где выполняется indexing, embedding и generation. Локальные embedding-модели и локальный vector store позволяют не отправлять документы наружу. Но если финальная генерация идет в облачную LLM, нужно понимать, какие chunks уходят в облако.

12. Когда RAG не поможет

RAG не является универсальным лекарством. Он решает проблему доступа к знаниям, но не все проблемы поведения агента.

Проблема	Почему RAG не решает	Что использовать
Мало или плохие данные	Нечего искать или документы шумные.	Data cleaning, OCR, разметка, нормализация, сбор базы.
Нужна сложная логика	RAG достает факты, но не заменяет workflow.	Agents, state machine, tools, business rules.
Нужен устойчивый стиль ответа	RAG хранит факты, а не манеры поведения.	Prompt engineering, personalization, fine-tuning.
Нужны действия во внешних системах	RAG сам по себе не создает встречи и не меняет задачи.	MCP/tools/API integrations.
Сканы и старые форматы	Текст может быть недоступен или плохо извлечен.	OCR, multimodal RAG, preprocessing.

13. Антипаттерны RAG

- Индексировать все подряд без очистки и metadata.
- Не учитывать даты документов и отвечать по устаревшей версии policy.
- Не показывать sources, из-за чего невозможно проверить ответ.
- Класть в prompt слишком много chunks, превращая ответ в шум.
- Доверять только vector similarity без reranking и без фильтров.
- Не проверять права доступа: пользователь может получить chunk из чужого проекта.
- Считать, что RAG заменяет fine-tuning, agents или business logic.
- Не защищаться от prompt injection внутри документов: документ может содержать вредную инструкцию для LLM.

14. Практическая цель недели

Судя по лекции и скринам презентации, практическая цель недели - научиться работать с embedders, реализовать поиск по базе знаний, подключить найденные chunks к работе агента и возвращать ссылки/источники.

14.1. MVP: RAG Knowledge Base Agent

Минимальный проект недели можно сделать как консольное или web-приложение на Python. Главное - показать не красоту интерфейса, а pipeline.

12. Положить несколько документов в папку data/docs.
13. Написать ingestion: чтение документов, очистка, chunking.
14. Сделать embeddings локально или через API.
15. Сохранить chunks и vectors в vector store: Chroma, FAISS, SQLite+vectors или простой in-memory MVP.
16. На запрос пользователя находить top-k chunks.
17. Опционально сделать reranking top-k -> top-n.
18. Собрать prompt: вопрос + найденные chunks + правило отвечать только по источникам.
19. Сгенерировать ответ и вывести sources.
20. Добавить простую оценку качества: thumbs up/down или тестовый набор вопросов.

14.2. Пример структуры проекта

```
rag-knowledge-agent/  
├── README.md  
├── .env.example  
├── requirements.txt  
├── data/  
│   └── docs/  
├── src/  
│   ├── ingest.py  
│   ├── chunking.py  
│   ├── embeddings.py  
│   ├── vector_store.py  
│   ├── retriever.py  
│   ├── reranker.py  
│   ├── prompt_builder.py  
│   ├── llm_client.py  
│   └── main.py  
├── eval/  
│   ├── questions.json  
│   └── evaluate.py  
└── results/  
    └── demo_answers.md
```

14.3. Что показать на видео

21. Показать документы в базе знаний.
22. Запустить ingestion и объяснить chunking/overlap.
23. Показать, что chunks индексируются в vector store.
24. Задать вопрос, на который обычная LLM могла бы ответить общо.
25. Показать найденные chunks и sources.
26. Показать финальный ответ с ссылками/источниками.
27. Показать пример, где reranking улучшает результат или хотя бы объяснить место reranking в pipeline.
28. Показать один negative case: если источников нет, агент честно говорит, что не нашел данных.

15. Корректировки и замечания к лекции

Фрагмент / проблема	Как корректнее понимать
RUG / RAC / "рак"	Это ошибка транскрипции. Корректный термин - RAG.
"RAG подгружает знания в базу знаний модели"	Не в веса модели. RAG подмешивает найденные chunks в prompt во время inference.
"Ollama - лучшая модель"	Ollama - runtime/model server. Через нее можно запускать разные модели, включая embeddings.
Нормализация делением на максимум	Для embeddings чаще важна L2-нормализация и cosine similarity; конкретика зависит от модели/vector store.
RAG vs MCP	MCP - протокол инструментов и интеграций; RAG - retrieval знаний. Они могут работать вместе.
RAG решит проблему плохих ответов	Только если данные качественные, retrieval точный, prompt построен правильно, а ответ проверяется по sources.

16. Checklist перед сдачей задания недели

- Есть папка с документами или мини-база знаний.
- Есть ingestion pipeline.
- Есть chunking с понятным размером chunk и overlap.
- Есть embeddings и vector store.
- Есть retrieval top-k.
- Есть prompt builder, который явно вставляет найденные chunks.
- Ответ содержит источники или хотя бы document_id/title/path.
- Есть fallback: если chunks не найдены, агент не выдумывает ответ.
- Есть хотя бы простая evaluation-таблица с контрольными вопросами.
- В README описано, как запустить проект и что показано на видео.

17. Готовый prompt для Cursor / Claude Code

Ниже - пример задания для code assistant, который можно адаптировать под свой проект.

Ты senior Python backend engineer. Сделай учебный проект RAG Knowledge Base Agent.

Цель:

- индексировать документы из data/docs;
- резать их на chunks с overlap;
- строить embeddings;
- сохранять chunks и vectors;
- по вопросу пользователя находить top-k релевантных chunks;
- опционально делать reranking;
- собирать prompt с найденным контекстом;
- отвечать только на основе sources;
- выводить ссылки/названия документов и chunk ids.

Требования:

- Python 3.11+;
- чистая структура src/;
- .env.example;
- README с запуском;
- без тяжелого web UI, достаточно CLI;
- если данных нет в источниках, агент должен честно сказать, что не нашел ответ;
- добавь eval/questions.json с 5 тестовыми вопросами;
- добавь results/demo_answers.md с примером вывода.

Сначала предложи план и структуру файлов. После утверждения реализуй код по шагам.

18. Короткий итог

RAG - это базовый паттерн для превращения LLM из “общего болтуна” в помощника, который отвечает по конкретной базе знаний. Но RAG требует инженерной дисциплины: хороших данных, chunking, metadata, retrieval, reranking, citations и evaluation.

Для AI Advent эта неделя важна потому, что RAG соединяет предыдущие темы курса: context management, memory/state, MCP/tools и агентную архитектуру. В итоге хороший агент должен уметь не только помнить состояние и пользоваться tools, но и доставать проверяемые знания из своей базы.

19. Приложение А. Пример контрольного набора для оценки RAG

Чтобы не проверять RAG только глазами на одном удачном вопросе, полезно сделать небольшой golden dataset. В учебном проекте достаточно 5-10 вопросов, но лучше сразу хранить их в JSON или CSV, чтобы потом автоматизировать regression-check.

Вопрос	Ожидаемый источник	Что должно быть в ответе	Что считается ошибкой
Какая политика code review в проекте?	docs/code_review_policy.md	Количество approval, SLA review, ссылка на policy.	Ответ “обычно в индустрии...” без ссылки на внутренний документ.
Как настроить CI/CD для KMP?	docs/kmp_cicd.md	Конкретные шаги для Kotlin Multiplatform и используемый CI.	Старый обзор CI/CD без ответа на вопрос.
Какие API разрешены в бесплатном режиме?	docs/billing_limits.md	Список разрешенных API и лимиты.	Выдуманные лимиты или ответ без источника.
Что делать, если документ устарел?	docs/doc_lifecycle.md	Правило deprecated/archived и куда смотреть.	Использование archived-документа как актуального.

Как использовать этот набор

29. Запустить ingestion и индексировать документы.
30. Для каждого вопроса получить top-k chunks.
31. Проверить, попал ли ожидаемый документ в top-k.
32. Сгенерировать ответ и проверить, что он опирается на найденные chunks.
33. Сохранить результат в results/eval_report.md.

20. Приложение В. Шаблон prompt для ответа по RAG

Один из главных источников ошибок - prompt, который не заставляет модель держаться найденных источников. Для учебного проекта лучше использовать жесткую формулировку.

SYSTEM:

Ты помощник, который отвечает только на основе предоставленных источников.

Если источников недостаточно, честно скажи: "В базе знаний не найдено достаточно данных".

Не используй общие знания модели для фактов, которые должны быть подтверждены документами.

В конце ответа всегда укажи Sources: document title + chunk id.

USER QUESTION:

{user_question}

RETRIEVED CONTEXT:

```
[chunk_id={chunk_id}, title={title}, updated_at={updated_at}, score={score}]
{chunk_text}
```

RESPONSE RULES:

1. Сначала дай прямой ответ.
2. Затем кратко объясни, на какие фрагменты опираешься.
3. Если sources конфликтуют, явно скажи о конфликте.
4. Если документ устарел или archived, не используй его как актуальный без предупреждения.

21. Приложение С. Мини-псевдокод RAG pipeline

Этот псевдокод не привязан к конкретной библиотеке. Его задача - показать архитектуру, которую можно реализовать через Chroma, FAISS, pgvector, Qdrant или даже in-memory MVP.

```
def ingest_documents(path):
    docs = load_documents(path)
    clean_docs = [clean_text(doc) for doc in docs]
    chunks = chunk_documents(clean_docs, chunk_size=800, overlap=120)
    vectors = embedding_model.embed([chunk.text for chunk in chunks])
    vector_store.upsert(chunks, vectors, metadata=True)

def answer_question(question):
    query_vector = embedding_model.embed_query(question)
    candidates = vector_store.search(query_vector, top_k=20, filters={"archived": False})
    best_chunks = reranker.rank(question, candidates, top_n=5)
    prompt = build_rag_prompt(question, best_chunks)
    answer = llm.generate(prompt)
    checked_answer = validate_faithfulness(answer, best_chunks)
    return checked_answer
```

Что здесь важно

- clean_text должен удалять мусор из PDF/HTML: футеры, дубли, навигацию, номера страниц.
- chunk_documents должен сохранять metadata: источник, заголовок, дату, section.
- search должен поддерживать фильтры, иначе старые документы будут мешать актуальным.
- validate_faithfulness может быть простым: проверка, что ключевые утверждения есть в retrieved chunks.

22. Приложение D. Разбор типичных ошибок в учебной реализации

Ошибка	Почему плохо	Как исправить
Ответ без sources	Нельзя проверить, откуда взялся факт.	Всегда выводить document title, path/url, chunk id.
Тор-к слишком маленький	Нужный документ может не попасть в prompt.	Тестировать k=5,10,20 и смотреть recall.
Тор-к слишком большой	Prompt становится шумным, модель смешивает темы.	Добавить reranking и брать top-n после rerank.
Нет fallback	Модель начинает выдумывать, когда retrieval пустой.	Если score ниже порога - честно говорить, что данных нет.
Все документы без даты	Невозможно отличить актуальное от старого.	Добавить metadata updated_at/status.
Индексируются скрины без OCR	В vector store попадает пустота или мусор.	Добавить OCR или не включать такие документы в MVP.

23. Приложение E. Как это связать с предыдущими неделями курса

Лекция 5 не отдельная тема, а продолжение всей архитектуры агента.

Неделя	Что изучали	Как используется в RAG-агенте
1	Prompting и параметры LLM.	Prompt builder, правила ответа, temperature для фактических ответов.
2	Context management и memory.	Не пихать всю базу в prompt, работать через chunks и retrieval.
3	Stateful agent.	Хранить state запроса, профиль пользователя, историю найденных sources.
4	MCP и tools.	Подключать внешние источники/Confluence/API как tools, а знания индексировать через RAG.
5	RAG.	Доставать знания из базы и отвечать с источниками.

24. Финальная формулировка, которую стоит запомнить

RAG - это инженерный способ заставить LLM отвечать не “из головы”, а по найденным документам. Он не заменяет агента, MCP, fine-tuning или бизнес-логику. Он закрывает конкретную проблему: как найти нужный кусок знаний и положить его в контекст модели так, чтобы ответ был проверяемым.

Хорошая демонстрация для этой недели - не просто “бот ответил на вопрос”, а доказательство pipeline: документы → chunks → embeddings → retrieval → reranking → prompt → ответ → sources → evaluation.